

Lab 3: Component Modelling & Architectural Pattern Selection

PES University · Dept. of CSE | Problem #65: Urban Rooftop Gardening Co-op App

Name: ALIFIYA SADIKOT SRN: PES1UG25CS841

Architecture Selection & Justification

1. Architectural Choice

Selected Style: Layered (Three-Tier) Architecture

The system is organised into three horizontal tiers: a Presentation Layer (Mobile/Web UI), a Business Layer (Garden Plot Manager, Community Hub, Produce Trade Board), and a Data Layer (Database Component). Each layer communicates only with the layer directly below it; components within the same Business Layer may also coordinate directly for tightly coupled domain operations (e.g., Garden Plot Manager triggers harvest alerts consumed by Community Hub).

2. Reasons for Selection

Reason 1 — Natural Tier Alignment

The app has clearly distinct concerns: user interaction (registering plots, viewing tips, placing trade requests), domain logic (computing harvest windows, moderating listings, broadcasting notifications), and persistent storage. These map directly onto three layers, keeping each tier's responsibilities cohesive and independently maintainable—the Produce Trade Board can add a bartering algorithm without touching the UI or database schema.

Reason 2 — Right-Sized Complexity for a Co-op App

A community co-op platform serves a bounded set of users and needs neither the independent-deployment overhead of Microservices nor the single-kiosk simplicity of Client–Server. Layered Architecture provides the structure needed for a multi-feature app (plot management, community hub, trade board) without the operational cost of inter-service communication, distributed tracing, and API gateways.

3. Security Advantage

The Data Layer is never directly accessible from the Presentation Layer. All client requests pass through the Business Layer, where authentication tokens are validated and role-based authorisation is enforced (e.g., only a Community Leader can publish announcements; only the plot owner can edit their yield log). This centralised gate prevents raw database queries from the UI, significantly reducing the attack surface for SQL injection and unauthorised data access.

4. Performance Benefit

The Business and Data layers are co-located on the same server, making Business-to-Data calls in-process method invocations rather than network round-trips. This keeps write latency low when a gardener logs a crop yield or a trade listing is persisted. For real-time features such as harvest alerts and surplus produce notifications, the architecture supports WebSocket or server-sent event channels between the server and the Mobile/Web UI, enabling push-based updates to gardeners and Community Leaders without polling overhead.
